

Teaching Theory of Computation in the STEM K-12 Curricula Through Impossibility and Undecidability Problems

Rafael del Vado Vírveda
Departamento de Sistemas Informáticos y Computación
Universidad Complutense de Madrid
Madrid, Spain
rdelvado@sip.ucm.es

Abstract

Theoretical Computer Science is a challenging subject to motivate and teach, particularly for students at the beginning of their Computer Science education. Introducing this topic at the high school level within the framework of STEM K-12 curricula (Science, Technology, Engineering, Mathematics), and linking real-world problems from these areas to core theoretical concepts, may make theory of computation more approachable for students prior to college. This paper presents a pedagogical approach for integrating theory of computing topics into STEM K-12 courses. It includes a set of classroom activities based on the Collatz conjecture, Feynman's infinite ladder of resistances, and polyomino tiling inspired by Penrose's cosmological models. These activities combine computational experimentation in Python with conceptual discussion to help students distinguish between different sources of computational limits, including chaotic unpredictability, mathematical impossibility, and algorithmic undecidability. While the problems and activities are primarily embedded in mathematics and physics courses, they are clearly connected to computational challenges across other STEM K-12 disciplines. To evaluate the effectiveness of this methodology, we analyze experimental data showing that introducing theoretical computing concepts, particularly those related to impossibility and undecidability, can enhance students' academic performance and motivation, compared to traditional K-12 programming courses.

CCS Concepts

• **Social and professional topics** → **Computational science and engineering education; Computational thinking; K-12 education**; • **Theory of computation** → **Computability**; • **Applied computing** → **Mathematics; Physical Sciences; Engineering**.

Keywords

Theoretical computing; impossibility; undecidability; STEM; K-12

ACM Reference Format:

Rafael del Vado Vírveda. 2026. Teaching Theory of Computation in the STEM K-12 Curricula Through Impossibility and Undecidability Problems. In *Proceedings of the 31st ACM Conference on Innovation and Technology in Computer Science Education V. 1 (ITiCSE 2026)*, July 10–15, 2026, Madrid, Spain. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3803400.3809304>



This work is licensed under a Creative Commons Attribution 4.0 International License. *ITiCSE 2026, Madrid, Spain*

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2634-7/2026/07
<https://doi.org/10.1145/3803400.3809304>

1 Introduction and Motivation

The integration of Theoretical Computer Science (TCS) into introductory college-level CS curricula is a subject of active debate [23]. However, there is an equally urgent need to explore how TCS concepts can be meaningfully introduced at the High School (HS) level within the framework of STEM K-12 curricula (Science, Technology, Engineering, Mathematics) [4, 12, 30]. Recent educational shifts emphasize a transition from syntax-driven coding toward broader computational thinking [28, 57], where TCS can play a pivotal role. By offering tools to explore the theoretical limits of computation and the nature of algorithmic reasoning, TCS provides a deeper analytical foundation than programming alone. For instance, while HS students are typically taught that mathematical statements are binary (either true or false) they are seldom exposed to the concept of inherent *undecidability*. Statements such as the Continuum Hypothesis, independent of ZFC axioms [29], demonstrate that some truths are unprovable within a given theoretical framework. Similarly, while students learn what software can do, they are rarely introduced to what it cannot do, such as the *impossibility* of an algorithm that detects all potential bugs in other programs [52].

All these theoretical boundaries are not mere abstract curiosities; they are foundational principles increasingly relevant to domains such as physics [60] and mathematics [45]. Despite this relevance, current pre-university CS education remains heavily focused on basic imperative programming [38, 39], often neglecting core questions regarding program efficiency, complexity, and algorithmic limits. To address this gap [22], we propose a pedagogical approach that bridges TCS and the STEM K-12 curricula through Python-based programming activities centered on classical impossibility and undecidability results. Our contribution is twofold. First, we present a set of educational activities that leverage intellectually rigorous and conceptually challenging TCS problems to motivate students within existing HS mathematics and physics courses. Second, we describe an approach that helps bridge the gap between introductory HS programming and the more abstract TCS content students typically encounter in later university-level coursework.

This work builds on a series of prior studies that introduced impossibility and undecidability as pedagogical TCS tools in university mathematics [17] and physics [18], and later partially explored their adaptation to various HS contexts [15, 16, 19]. While these contributions demonstrated the potential of this TCS approach, they remained fragmented across disciplines and educational levels. The present paper advances this line of work by providing a unified and scalable framework for integrating TCS concepts into the broader STEM K-12 curricula, supported by new classroom-tested activities in Python and a rigorous empirical and statistical evaluation.

Table 1: Last two courses of the HS Mathematics curricula with impossibility and theoretical computing questions [16, 17]

HS Course	HS Maths Curriculum	Impossibility Results	Computability Questions	Complexity Questions
1 st (EV1)	Irrational numbers.	$\sqrt{2}$ is not a rational number.	Irrational number in a computer.	Estimating π using <i>Monte Carlo</i> .
1 st (EV2)	Algebraic equations.	Equation of the fifth degree.	Diophantine equations.	SAT / Boolean Satisfiability.
1 st (EV3)	Exponentials and series.	Chessboard legend.	Exponential growth.	Towers of Hanoi, Chess, Go.
1 st (EV4)	2D Geometry.	Problems of Greek antiquity.	Constructing a regular polygon.	<i>Mathekreis</i> and <i>Mathe Quadrat</i> .
2 nd (EV1)	Linear algebra (matrices).	Non-commutative operations.	Matrix mortality problem.	Fastest algorithms for matrices.
2 nd (EV2)	3D Geometry.	Independence of postulates.	Axiomatic systems for geometry.	3D Tiling puzzles.
2 nd (EV3)	Differentiation & Integration.	Non-integrable functions.	Decision of the integrability.	Numerical integration.
2 nd (EV4)	Probability & Statistics.	Integral of normal distributions.	Improper integral convergence.	The Galton board.

2 Related Work, Contributions, and Approach

The integration of TCS at the K-12 level aligns with the “Great Principles of Computing” framework [21], which advocates for teaching computing as a science rather than just a toolset. Much of the early work on TCS education in secondary schools emerged during the late 1980s [43, 49] and early 1990s [25, 46]. More recent initiatives can be found in [6, 7, 30, 35], and a comprehensive systematic review of TCS in secondary education over the past two decades is presented in [33]. However, most of the literature reporting results across individual STEM areas, particularly in mathematics [10, 40, 45, 53], remains too complex or advanced to be directly adopted in K-12 classrooms, despite its significant pedagogical potential. Similarly, while widely cited references on impossibility in physics [5, 34, 50, 58] are conceptually rich, they often lack explicit algorithmic content that connects to computability and TCS topics relevant for K-12 physics curricula.

Despite these challenges, the body of related work shows a growing interest in integrating TCS into K-12 STEM education. For example, resources such as the New Zealand Computer Science Field Guide [7] offer interactive and engaging ways to teach TCS, making it more accessible for young learners. However, many of these efforts rely on highly motivated students or those with prior exposure to CS, which can exacerbate learning disparities. Furthermore, the majority of these programs remain pilot studies with limited scalability [33], and there is little longitudinal research on knowledge retention or conceptual transfer. Therefore, broader curricular integration and instructional strategies are needed to maximize TCS’s educational potential across diverse student populations.

Our contribution to this landscape lies in presenting TCS concepts, particularly those centered on mathematical impossibility and algorithmic undecidability, through Python-based activities that are directly applicable to K-12 STEM curricula. Unlike alternative approaches that focus on visualizer tools [24], our approach utilizes the HS students’ existing mathematical and scientific maturity in STEM tracks to introduce formal limits, a strategy that echoes the “CS Unplugged” philosophy but adapts it for HS using Python-based simulations [6]. Our approach begins at the HS level and leverages existing curricular content, specifically impossibility results already embedded in HS mathematics and physics [18]. Around these limitations, we construct associated undecidability questions and concrete Python programming activities that STEM K-12 students can explore, without having to present complex TCS concepts on abstract models of mathematics (see **Table 1**) and without delving into formal models of physics (see **Table 2**).

3 Early Motivational Examples

In this section, we present illustrative examples used exclusively for motivation and conceptual introduction; these examples are not part of the evaluated educational interventions described in later sections. Their purpose is to provide intuitive entry points to fundamental ideas from TCS before introducing the educational activities that were formally assessed. We use the term *limits of computation* in a broad educational sense to refer to several different phenomena that arise in the STEM context. These include problems that are impossible to solve in closed form, problems that are computationally intractable, systems whose dynamics are unpredictable due to chaos, problems that are algorithmically undecidable, and statements that are independent of a given axiomatic system. Although these notions are distinct in formal terms, presenting them together in an educational STEM K-12 context helps students appreciate that limits to computation arise in multiple ways.

During the final years of HS mathematics, students are introduced to the impossibility that some functions cannot be integrated in terms of elementary functions. For example, the following integrals have no elementary antiderivatives [27]: $\int e^{-x^2} dx$, $\int \frac{\sin(x)}{x} dx$, and $\int \sin(x^2) dx$. As a result, HS students must resort to approximations using calculators, statistical tables, or computer programs to estimate values such as $\frac{1}{\sqrt{2\pi}} \int_a^b e^{-\frac{x^2}{2}} dx$, the cumulative distribution function of the standard normal distribution. A contemporary computational implication of this mathematical impossibility arises in Bayesian inference, with direct relevance to machine learning and deep learning [8]. In many such models, the posterior distribution cannot be computed using Bayes’ theorem because the required integral does not admit an analytical solution.

Naturally, students begin to wonder whether there exists a computer program capable of determining whether an arbitrary function admits an elementary integral. As shown by Richardson [48], the problem of deciding whether a function has an elementary antiderivative is undecidable: there is no general algorithm that can answer this question for all functions. This insight often surprises students, who may assume that the absence of such a program is due to a lack of current knowledge rather than a fundamental theoretical limitation. It provides a gateway to discussing the limitations of algorithms and the idea that some problems are beyond the reach of computation. Using Cantor’s diagonal argument [54], students can understand that the set of all computer programs is countable, while the set of functions we may want to compute (e.g., real-valued functions from $\mathbb{N}$ to $\mathbb{N}$) is uncountable. This cardinality gap makes

Table 2: Last two courses of the HS Physics curricula with impossibility results and undecidability problems [18, 19]

HS Course	HS Physics Curriculum	Impossibility and Undecidable Problems
1 st (EV1)	Newtonian dynamics.	Undecidability of generalizations of the Collatz conjecture / Impossibility of the three-body problem.
1 st (EV2)	Oscillations and pendulums.	Chaotic motion of a double pendulum / Undecidability, unpredictability, and chaos (computing fractals).
1 st (EV3)	Thermodynamics.	Impossibility of a perpetual motion machine / Maxwell’s Demon impossibilities.
1 st (EV4)	Light and optics.	Reverse ray tracing problem / Non-computable light paths.
2 nd (EV1)	Electromagnetics.	Feynman’s infinite electrical circuit / Impossibility of irrational number representation in computers.
2 nd (EV2)	Hydrodynamics.	Impossibility of the floating rubber ducks problem / Undecidability of fluids (river of lava from a volcano).
2 nd (EV3)	Quantum physics.	Impossibility of the Heisenberg’s uncertainty principle / Undecidability of the spectral gap (intuitively).
2 nd (EV4)	Cosmology.	Undecidability of the polyomino problem (toy universe of Penrose) / Undecidability in tiling problems.

it inevitable that some functions, including many arising in calculus and physics, are not computable. From this perspective, the concept of undecidability becomes intuitively accessible: there are more problems than algorithms, and therefore, some problems cannot be solved by any algorithm, no matter how advanced. The integration problem offers a compelling illustration of this limitation.

This foundational idea opens the door to understanding undecidable phenomena in physics. HS students studying classical mechanics (Newton’s dynamics, dynamical systems, oscillations and pendulums) often develop the mistaken belief that physical systems are fully predictable, given sufficient data and precision [2]. However, certain dynamical systems exhibit behavior that is not only chaotic but also provably undecidable [14, 48]. One such system is the classical *three-body problem*. HS students simulate the motion of three mutually interacting bodies using Python, observing chaotic behavior due to extreme sensitivity to initial conditions (see **Figure 1**). Another instructive example is the *double pendulum*, a mechanical system composed of one pendulum suspended from another. When set in motion with sufficient energy, it exhibits chaotic and unpredictable behavior. This system can be easily built in a HS physics lab, and its motion can be simulated and analyzed computationally. Once students have experimented with it physically, they can implement simulations in Python to illustrate chaotic dynamics and practical unpredictability. Small differences in initial conditions θ_0 rapidly lead to dramatically different trajectories, making long-term prediction extremely difficult (see **Figure 2**).

These limitations are not restricted to cosmological or oscillation phenomena, but arise across a wide range of STEM contexts, including molecular modeling, complex ecosystems, and the dynamics of crowds and traffic flows. This broader perspective helps students recognize that computational limits are a general feature of scientific modeling, rather than a property of isolated theoretical problems. This makes such examples particularly effective for introducing TCS concepts within STEM K–12 education. Beyond unpredictability arising from chaotic dynamics, recent work provides experimental evidence that incomputability can also manifest in physical systems [1]. All these STEM problems serve as intuitive gateways to the broader and more abstract concepts of impossibility and undecidability. They demonstrate that limitations in computation are not confined to abstract mathematical or physical settings.

4 Study Context, Methodology, and Design

Our pedagogical approach was implemented over two consecutive academic years in the final two years of a STEM-oriented public HS

with a mathematics and physics curriculum. Each academic year was structured into four quarterly evaluation periods (EV), within which the proposed activities were integrated as short instructional interventions (see **Tables 1–2**). Participants were students aged 15–18 with no systematic prior exposure to TCS beyond basic programming. All participating students had previously completed an introductory programming module using Python as part of their computing curriculum. The activities described in this study did not require advanced programming techniques; instead, they asked students to combine these familiar constructs to explore mathematical or physical processes computationally. When necessary, short code templates were provided so that the focus remained on conceptual exploration rather than on programming syntax. The activities were embedded within the standard STEM K–12 curricula and did not correspond to specialized or elective CS tracks. All participants provided informed consent for the use of anonymized data for research purposes and the study was conducted in accordance with the institution’s ethical guidelines for educational research.

Tables 1–2 present a broader set of potential classroom activities illustrating different aspects of computational limits. Our experimental study focuses on three activities presented in [19] designed to introduce core TCS concepts (computability, undecidability, and algorithmic limits) through familiar STEM contexts: (i) Feynman’s infinite circuit (computable vs. non-computable numbers), (ii) the Collatz conjecture (unpredictability and undecidability), and (iii) the polyomino tiling problem inspired by Penrose’s cosmological model (undecidability in geometric and cosmological settings). We structure these Python activities along a progression from mathematical impossibility to computational unpredictability, and finally algorithmic undecidability. The remaining activities in **Tables 1–2** are proposed as curricular extensions and were not part of the controlled evaluation. The selected Python activities are formally described and statistically analyzed in the following subsections.

All activities were integrated as short instructional interventions within regular classroom instruction. To assess their educational value, we conducted controlled experiments involving a total of 80 students. Students were randomly assigned to one of two groups: a *control group* ($n = 40$) followed the standard curriculum without the programming-based activity, and an *experimental group* ($n = 40$) completed the corresponding Python-based activity. The same cohort of students participated in the three activities over the course of the academic year. Because the same cohort participated in multiple interventions, a potential carryover effect cannot be completely excluded. However, the different activities were separated in time.

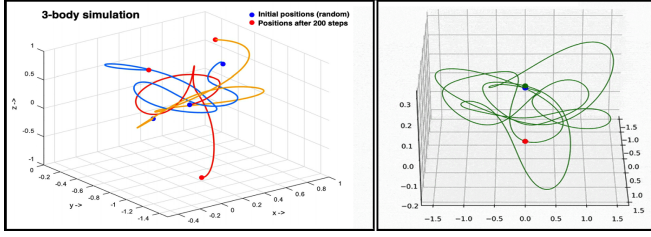
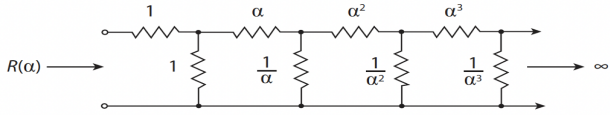


Figure 1: Three-body problem [2].

HS student learning was assessed using concept-focused post-tests targeting understanding of computability, undecidability, and algorithmic limits rather than mathematics or physics content. A detailed description of this assessment is provided in Section 5.

4.1 Infinite Circuits and Computable Numbers

One of the first mathematical impossibility results encountered by HS students is the proof that $\sqrt{2}$ is non-rational [54]. This result introduces the idea that irrational numbers have non-terminating, non-repeating decimal expansions and therefore cannot be exactly represented in a computer's memory. HS students learn to approximate such numbers using numerical methods such as *continued fractions*. A compelling example arises in the context of electric circuits, where we examine the input resistance of an infinite “ladder” circuit, an example popularized by Feynman [42]. The circuit consists of repeating segments of horizontal and vertical resistors:



where α represents the multiplier coefficient by which the horizontal resistances decrease in value from left to right, and $\frac{1}{\alpha}$ represents the multiplier coefficient by which the vertical resistances increase in value in the same direction. We denote $R_k(\alpha)$ the input resistance of the *finite* version in which we distinguish k steps (a step is a horizontal resistance plus a vertical resistance). In the last HS physics course, our students learn that $R_1(\alpha) = 2$ and $\lim_{k \rightarrow \infty} R_k(\alpha) = R < 2$. This input resistance R for the *infinite* ladder circuit can be expressed as an infinite continued fraction $R(\alpha)$ (see the image below) combining the well-known formula for parallel resistances, taught in HS physics: $R_{\text{parallel}} = \frac{1}{\frac{1}{R_1} + \frac{1}{R_2} + \dots + \frac{1}{R_k}}$.

There is a theorem [42] that proves that this particular value of R is irrational, but computers can only represent irrational numbers approximately. Therefore, there's no hope of expressing the complete value of R in a finite machine. However, with a simple Python program students can calculate their values ($k \geq 2$) progressively:

```
def R(alpha, k, n = 0):
    if n == 0 : return 1 + 1 / R(alpha, k, 1)
    if n == 1 : return 1 + 1 / R(alpha, k, 2)
    if n == 2 * k - 2 :
        a = alpha ** (k - 1)
        return a + 1 / a
    a = alpha ** (n // 2)
    return a + 1 / R(alpha, k, n + 1)
```

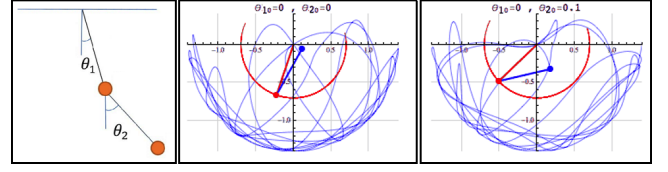
$$R(\alpha) = 1 + \frac{1}{1 + \frac{1}{\alpha + \frac{1}{\alpha^2 + \frac{1}{\alpha^3 + \dots}}}}$$


Figure 2: Double pendulum [51].

Beyond its value as an engaging puzzle, Feynman's infinite ladder of resistances provides a natural entry point to discussing computable and non-computable numbers. Our practical activity helps students understand that the final resistance $R(\alpha)$ is an irrational number, and more precisely, a *computable number*, since it can be approximated to any desired degree of precision by a terminating algorithm. However, most real numbers (including irrational ones) are *non-computable*, as there are uncountably many reals but only countably many programs. This leads naturally to the concept of undecidability within the context of number representation: while some irrational numbers can be computed algorithmically (like $R(\alpha)$), the vast majority cannot. The existence of such non-computable numbers reflects deep limits of what can be encoded or represented within any algorithmic system. There are recent facts in physics and computing that can be shown to our HS students that follow from the existence of non-computable numbers [47, 60], highlighting that the boundary between mathematics, CS, and physical theory remains an active area of research [41]. For example, the Feynman's infinite ladder can be further extended to more complex and rich settings, such as *fractal circuits* [3, 11].

To assess the educational value of this activity from [19], HS students in the experimental group were prompted to reflect on questions such as: “Why can't a computer represent R exactly, even though we can compute it with arbitrary precision?”. We employed a *Welch's t-test* (with significance level $\alpha = 0.05$) to compare the mean test scores between groups under the hypotheses: H_0 , no difference in mean scores, and H_1 , the experimental group has a higher mean score. The statistical analysis in Table 3 revealed a significant improvement in the experimental group's understanding of computability:

Table 3: Test scores for Feynman's infinite circuit activity

Group	Mean Score	Standard Deviation	Sample Size
Control	5.24	1.11	40
Experimental	7.13	0.93	40

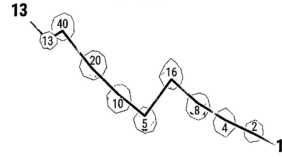
The resulting t -statistic of 5.88 and p -value of 1×10^{-6} led us to reject the null hypothesis H_0 , confirming a significant improvement in conceptual understanding. The smaller standard deviation in the experimental group suggests that the activity benefitted a broad range of students, not just those with stronger math backgrounds.

4.2 Collatz Conjecture and Hailstones

The *Collatz Conjecture* is a remarkably simple “impossibility” problem with profound implications for computability and unpredictability [37]. It begins with any positive integer n . If n is even, divide

it by two; if odd, multiply it by three and add one. This process is repeated recursively. The conjecture states that regardless of the starting value, the sequence will eventually reach 1. This iterative process is easily programmed by HS students to search for counterexamples:

```
def collatz_sequence(n):
    sequence = [n]
    while n != 1:
        if n % 2 == 0: n = n // 2
        else: n = 3 * n + 1
        sequence.append(n)
    return sequence
```



Although it appears to be a mere mathematical curiosity, with no obvious real-world applications, there is an interesting connection between the Collatz conjecture and a quite common, damaging, physical and meteorological phenomenon: *hailstorms* [19]. Collatz sequences are called "hailstone sequences", as they can bounce up-down like hailstones in a cloud (see the image above). This physical analogy makes the abstraction more relatable to HS students and facilitates interdisciplinary discussion, connecting number theory, dynamical systems, and computational limits. Despite its simplicity, the Collatz problem remains unsolved since the 1930s. In 1972, Conway demonstrated that a generalized version of the Collatz process with interesting real-world physical applications (fluids, ecosystems, and climate) is algorithmically undecidable [29, 36]: $f(n) = a_i \cdot n + b_i$ if $n \bmod m = i$, for all $i = 0, \dots, m-1$. The Collatz conjecture provides a striking example of a STEM problem that is extremely easy to state yet remains unsolved, and thus has the potential to motivate and attract young people towards theoretical computation [13]. Despite extensive computational evidence supporting the conjecture, no general proof or refutation is currently known [55]. In our classroom activity, students used simple Python programs to explore Collatz sequences and observe empirical regularities. This exploration serves as an entry point to discussing the question of whether simple dynamical rules might encode computationally complex or even undecidable behaviors.

A controlled experiment involving the same students was conducted, comparing the performance of students who completed the activity with those who did not. Students in the experimental group implemented the Python program to generate Collatz sequences, count steps to reach 1, and determine the maximum value reached. The students are then asked to reflect on conceptual questions such as: "Why is it so difficult to predict whether a sequence will reach 1? Could a computer eventually solve this problem? If not, why?". Statistical analysis in **Table 4** via *Welch's t-test* demonstrates again an improvement among students who participated in the activity:

Table 4: Test scores in the Collatz conjecture activity

Group	Mean Score	Standard Deviation	Sample Size
Control	5.41	1.29	40
Experimental	7.58	1.02	40

The test yielded a t -statistic of 8.11 with a p -value of 1×10^{-8} , leading us to reject the null hypothesis and conclude that the Collatz conjecture, with its simple rules and complex behavior, proved to be an effective tool for introducing students to undecidability.

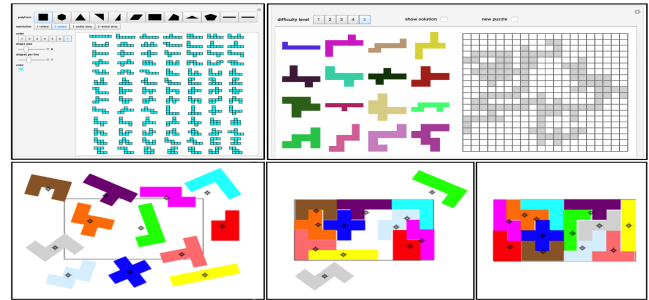


Figure 3: Polyomino puzzles and tilings [56, 59].

4.3 Polyomino Problem and Cosmology

Tiling problems provide a visually intuitive entry point into the notions of mathematical impossibility and algorithmic undecidability. Learners of all ages enjoy solving puzzles involving geometric tiles, such as covering a plane with *polyominoes*, shapes formed by joining unit squares edge-to-edge [26]. A classical first example is the *mutilated chessboard problem* [17], where removing two opposite corners of an 8×8 standard chessboard makes it impossible to tile with *dominoes*. Each domino covers one black and one white square, but the removal disrupts this balance. This illustrates how simple local constraints can lead to global impossibility, providing an intuitive bridge toward more complex algorithmic questions about tilings [31]. These tasks are not only accessible but also mathematically rich, and they connect surprisingly well with current undecidability results in physics [60]. One compelling pedagogical link comes from Roger Penrose, who proposed a model of *cosmology* in which the evolution of a "toy universe" is represented by finite sets of polyominoes [44]. In this model, each state at discrete time step n is given by a pair of polyomino sets (S_n, S_{n+1}) , and the evolution follows a deterministic rule based on a global tiling property: if the first set can tile the plane, the system advances to the next set in the sequence; otherwise, the sets are interchanged. Thus, successive states of a discrete spacetime structure can be interpreted through tiling-like constructions [32, 56].

While the full physical theory is far beyond the scope of secondary education, presenting a simplified version of this STEM motivational context helps students see how abstract mathematical structures, such as tilings, may appear in attempts to model fundamental aspects of the Universe. Students begin by experimenting with finite configurations of polyominoes and formulating conjectures about whether a given set of tiles can cover the plane using [59] (see **Figure 3**). This naturally leads to the deeper question of whether a general algorithm exists to decide the tiling property. While students can easily experiment with small grids and specific tile sets, the general problem (deciding whether an arbitrary finite set of polyominoes can tile the infinite plane) is undecidable [9]. There exists no algorithm that can answer this question for all possible tile sets. Then, although the evolution of this little universe is deterministic, it is non-computable. This result not only bridges TCS and cosmology, but also highlights how undecidability can emerge even in problems that seem purely visual or combinatorial. Recent research continues to push the boundary by identifying undecidability results for increasingly small sets of tiles [20].

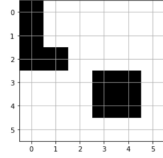
To assess the impact of this activity of [19], we conducted an experiment with the same cohort of HS students, again divided into control and experimental groups. The experimental group implemented a Python-based simulation to attempt tiling a 6×6 grid:

```
L_tromino = np.array([[1, 0], [1, 0], [1, 1]])
square = np.array([[1, 1], [1, 1]])

grid = np.zeros((6, 6), dtype=int)

def place_tile(grid, tile, top_left):
    x, y = top_left
    tile_h, tile_w = tile.shape
    if x + tile_h <= grid.shape[0] and
       y + tile_w <= grid.shape[1]:
        grid[x:x+tile_h, y:y+tile_w] += tile
    return grid

grid = place_tile(grid, L_tromino, (0, 0))
grid = place_tile(grid, square, (3, 3))
```



Students described their algorithmic strategies and implications: “Why might it be impossible to generalize your method to infinite grids? What does this tell us about the limits of automated reasoning?” Welch’s *t*-test showed in **Table 5** an improvement in the experimental group’s understanding of undecidability:

Table 5: Statistics of test scores for the polyomino problem

Group	Mean Score	Standard Deviation	Sample Size
Control	5.62	1.39	40
Experimental	7.82	1.08	40

The *t*-statistic of 7.59 and *p*-value of 3×10^{-8} confirm a strong effect: hands-on exploration of undecidability through geometric puzzles not only enhances student understanding but also improves their ability to reason abstractly about the boundaries of computation.

5 Discussion, Limitations, and Implications

The assessment methodology was designed to explicitly evaluate students’ ability to transfer their understanding to core TCS concepts, such as recognizing the relevance of the Halting Problem to questions of termination in simulations of the Collatz sequence, rather than to assess standard maths or physics curriculum content. Open-ended responses were evaluated using a predefined rubric, applied across all activities, with total scores normalized to a 0–10 scale (see **Figure 4**). The experimental results suggest that embedding core TCS concepts within the standard STEM K–12 curricula is both feasible and pedagogically beneficial for fostering computational thinking. One particularly effective aspect was the use of well-known physical problems, such as the classical three-body problem (e.g., Earth–Moon–spacecraft systems like *Artemis II*), as conceptual “hooks” for introducing abstract TCS ideas.

Several limitations should be considered when interpreting these results. First, the participants were HS students following a strong STEM-oriented curriculum, with relatively advanced backgrounds in mathematics and physics. This prior preparation may have made them more receptive to abstract theoretical ideas than the general K–12 student population. Second, the intervention was implemented over a limited number of instructional sessions, and no longitudinal

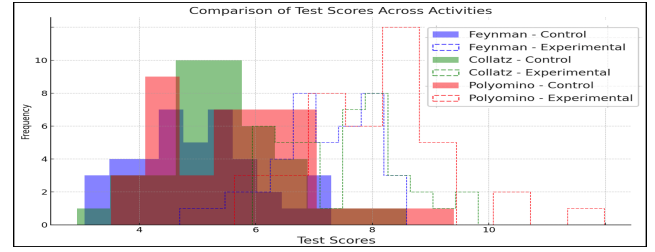


Figure 4: Comparison histogram of test scores.

data were collected. As a result, the persistence of students’ intuitions about impossibility and undecidability beyond the immediate instructional context remains an open question.

Despite these constraints, the statistical study suggests several pedagogical implications for educators interested in replicating or adapting this approach. First, TCS concepts should not be taught in isolation, but rather integrated into existing STEM courses where natural conceptual connections already exist. Second, the use of a familiar programming language (in this case, Python) proved effective as a didactic vehicle for experimentation and simulation. Additionally, although our experimental study was conducted in a relatively strong STEM context, similar activities in **Tables 1–2** could potentially be adapted for more general STEM K–12 settings.

6 Conclusions and Future Work

This paper has presented a set of Python activities designed to introduce foundational TCS concepts through impossibility and undecidability problems embedded within the standard STEM K–12 curricula. By grounding abstract notions such as non-computability, undecidability, and algorithmic limits in familiar and real-world mathematical and physical contexts, the proposed approach aims to make core ideas from TCS accessible and meaningful to pre-university students. Across the three evaluated activities from [19], statistical analyses showed a consistent and significant improvement in the experimental groups (as shown in **Tables 3–5**), with score distributions shifted toward higher values (as can be seen in **Figure 4**). These results suggest that embedding basic TCS concepts within familiar STEM contexts can enhance students’ conceptual understanding of fundamental limits of computation. By encountering examples ranging from chaotic dynamics to undecidable tiling problems, students begin to appreciate that the limits of computation are themselves a rich and fascinating subject of study, extending to real-world systems (e.g., *Artemis II* mission dynamics). Introducing these ideas earlier in STEM education may help broaden students’ understanding of maths, physics, and CS.

Future work will address limitations outlined in the previous sections. First, we aim to develop a standardized suite of Python-based notebooks for STEM education, integrating automated feedback to support HS students in exploring the limits of computation through hands-on experimentation. Second, we plan to incorporate delayed assessments in order to evaluate long-term retention of key TCS concepts and students’ ability to apply them in novel contexts. Third, the proposed activities will be replicated in additional educational settings, including HS with different curricular structures and student populations, to better assess their generalizability.

References

- [1] José Manuel Agüero, Cristian S. Calude, Michael J. Dinneen, Arkady Fedorov, Anatoly Kulikov, Rohit Navarathna, and Karl Svozil. 2024. How Real is Incomputability in Physics? *arXiv preprint arXiv:2311.00908* (2024).
- [2] A. Aguirre, Z. Merali, and D. Sloan. 2021. *Undecidability, Uncomputability, and Unpredictability*. Springer International Publishing. <https://books.google.es/books?id=p00uzgEACAAJ>
- [3] Eric Akkermans, Joe P. Chen, Gerald V. Dunne, Luke G. Rogers, and Alexander Teplyaev. 2018. Fractal AC circuits and propagating waves on fractals. *arXiv preprint arXiv:1507.05682* (2018).
- [4] Michal Armoni. 2011. Looking at Secondary School CS Education Through the Lens of Theoretical Computer Science. *ACM Inroads* 2, 2 (2011), 24–27.
- [5] John D. Barrow. 1999. *Impossibility: The Limits of Science and the Science of Limits*. Oxford University Press.
- [6] Tim Bell, Jason Alexander, Isaac Freeman, and Mick Grimley. 2009. Computer Science Unplugged: School Students Doing Real Computing without Computers. *New Zealand Journal of Applied Computing and Information Technology* 13, 1 (2009), 20–29.
- [7] Tim Bell, Caitlin Duncan, Sam Jarman, and Heidi Newton. 2014. Presenting Computer Science Concepts to High School Students. *Olympiads in Informatics* 8 (2014), 3–20.
- [8] Shai Ben-David, Pavel Hruš, Shay Moran, Amir Shpilka, and Amir Yehudayoff. 2019. Learnability can be undecidable. *Nature Machine Learning* 1, 1 (2019), 44–48. <https://doi.org/10.1038/s42256-018-0002-3>
- [9] Robert Berger. 1966. The Undecidability of the Domino Problem. *Memoirs of the American Mathematical Society* 66 (1966).
- [10] Lenore Blum, Felipe Cucker, Michael Shub, and Steve Smale. 1998. *Complexity and Real Computation*. Springer-Verlag, Berlin, Heidelberg.
- [11] Joe P. Chen, Luke G. Rogers, et al. 2017. Power dissipation in fractal AC circuits. *Journal of Physics A* 50, 45 (2017).
- [12] Computer Science Teachers Association. 2017. *CSTA K-12 Computer Science Standards*. CSTA. <https://csteachers.org/standards>
- [13] Journal Contributors. 2025. A Novel Approach to the Collatz Conjecture with Petri Nets. *Information* 16, 9 (2025). <https://doi.org/10.3390/info16090745>
- [14] Newton da Costa Jr and Francisco Doria. 1991. Undecidability and incompleteness in classical mechanics. *International Journal of Theoretical Physics* 30 (08 1991), 1041–1073. <https://doi.org/10.1007/BF00671484>
- [15] Rafael del Vado Virseda. 2019. Computability and Algorithmic Complexity Questions in Secondary Education. In *CompEd 2019*. 51–57. <https://doi.org/10.1145/3300115.3309507>
- [16] Rafael del Vado Virseda. 2020. From the Mathematical Impossibility Results of the High School Curriculum to Theoretical Computer Science. In *Koli Calling '20: 20th Koli Calling International Conference on Computing Education Research, Koli, Finland, November 19–22, 2020*, Nick Falkner and Otto Seppälä (Eds.). ACM, 19:1–19:5. <https://doi.org/10.1145/3428029.3428038>
- [17] Rafael del Vado Virseda. 2021. Learning from the Impossible: Introducing Theoretical Computer Science in CS Mathematics Courses. In *SIGCSE '21: The 52nd ACM Technical Symposium on Computer Science Education, Virtual Event, USA, March 13–20, 2021*. ACM, 952–958. <https://doi.org/10.1145/3408877.3432475>
- [18] Rafael del Vado Virseda. 2023. Theoretical Computer Science Education from Impossibility and Undecidability Problems in Physics. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education, Volume 1, SIGCSE 2023, Toronto, ON, Canada, March 15–18, 2023*. ACM, 270–276. <https://doi.org/10.1145/3545945.3569742>
- [19] Rafael del Vado Virseda. 2025. Introducing Theoretical Computer Science in High School Physics Courses from Impossibility and Undecidability Problems. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 2, SIGCSE TS 2025, Pittsburgh, PA, USA, 26 February 2025 - 1 March 2025*. ACM, 1433–1434. <https://doi.org/10.1145/3641555.3705207>
- [20] Erik D. Demaine and Stefan Langerman. 2024. Tiling with Three Polygons is Undecidable. *arXiv preprint arXiv:2409.11582* (2024).
- [21] Peter Denning and Craig Martell. 2015. *Great Principles of Computing*. MIT Press.
- [22] Ryan E. Dougherty. 2025. Why Do Students Struggle in CS Theory Courses So Much? *ACM Trans. Comput. Educ.* 25, 1 (2025), 11:1–11:5. <https://doi.org/10.1145/3718347>
- [23] Ryan E. Dougherty et al. 2024. A Survey of Undergraduate Theory of Computing Curricula. In *Proceedings of the 2024 ACM Virtual Global Computing Education Conference V. 2, SIGCSE, Mohsen Dorodchi, Ming Zhang, and Stephen Cooper (Eds.)*. ACM. <https://doi.org/10.1145/3649409.3691070>
- [24] Ryan E. Dougherty et al. 2025. TheoryViz: A Visualizer Tool for Theory of Computing Concepts. In *Proceedings of the 56th ACM Technical Symposium on Computer Science Education V. 2, SIGCSE TS 2025, Pittsburgh, PA, USA, 26 February 2025 - 1 March 2025*. ACM, 1373–1374. <https://doi.org/10.1145/3641555.3705268>
- [25] Michael R. Fellows. 1993. Computer science and mathematics in the elementary schools. In *Mathematicians and Education Reform*. Amer. Math. Society.
- [26] Martin Gardner. 1960. Polyominoes. *Scientific American* 203, 5 (1960), 104–116. “Mathematical Games” column introducing polyominoes to a wide audience.
- [27] Joaquín Hernández Gómez. 1998. Using nonelementary results in non-university courses. *Revista epsilon - Thales* 41, 2 (1998), 285–294.
- [28] Shuchi Grover and Roy Pea. 2013. Computational Thinking in K-12: A Review of the State of the Field. *Educational Researcher* 42, 1 (2013), 38–43.
- [29] Dan Gusfield. 2024. *Proven Impossible: Elementary Proofs of Profound Impossibility from Arrow, Bell, Chaitin, Gödel, Turing and More*. Cambridge University Press.
- [30] Orit Hazzan and Tami Lapidot. 2020. Teaching theoretical computer science in secondary schools. *Computer Science Education* 30, 2 (2020), 123–145.
- [31] Emmanuel Jeandel and Pascal Vanier. 2021. Tilings and undecidability: A survey. *Journal of Symbolic Logic* (2021).
- [32] Ed Pegg Jr. 2009. Polyform Explorer. *Wolfram Demonstrations Project* (2009). demonstrations.wolfram.com/PolyformExplorer/
- [33] Braz A. S. Junior, Simone A. C. Cavalheiro, and Luciana Foss. 2021. Theoretical Computer Science in Basic Education: A Systematic Review. *Proceedings of the Workshop on Education in Information Technology (WEIT)* (2021).
- [34] Michio Kaku. 2008. *Physics of the Impossible: A Scientific Exploration Into the World of Phasers, Force Fields, Teleportation, and Time Travel* (1 ed.). Doubleday.
- [35] Laura Korte, Stuart Anderson, Helen Pain, and Judith Good. 2007. Learning by Game-building: A Novel Approach to Theoretical Computer Science Education. In *Proceedings of the 12th Annual SIGCSE Conference on Innovation and Technology in Computer Science Education (ITiCSE '07)*. ACM Press, New York, NY, USA, 53–57. <https://doi.org/10.1145/1268784.1268802>
- [36] Stuart A. Kurtz and Janos Simon. 2007. The Undecidability of the Generalized Collatz Problem. *Theory and Applications of Models of Computation* (2007), 542–553. https://doi.org/10.1007/978-3-540-72504-6_49
- [37] Jeffrey C. Lagarias. 2010. The $3x+1$ Problem: An Overview. *The Ultimate Challenge: The $3x+1$ Problem* (2010), 3–29.
- [38] Hans P. Langtangen. 2012. *A Primer on Scientific Programming with Python* (5th ed.). Springer Publishing Company, Incorporated.
- [39] Svein Linde and Hans P. Langtangen. 2016. *Programming for Computations - Python: A Gentle Introduction to Numerical Simulations with Python* (1st ed.). Springer Publishing Company, Incorporated.
- [40] José Luis Montaña and Luis M. Pardo. 2013. *Recent Advances in Real Complexity and Computation*. Vol. 604. Contemporary Mathematics.
- [41] Anna Muranova. 2022. On the effective impedance of finite and infinite networks. *Potential Analysis* 56 (2022), 697–721.
- [42] P. Nahin. 2011. Number-Crunching: Taming Unruly Computational Problems from Mathematical Physics to Science Fiction. *Princeton University Press* (2011).
- [43] Seymour Papert. 1980. *Mindstorms: Children, Computers, and Powerful Ideas*. Basic Books, Inc., New York, NY, USA.
- [44] R. Penrose. 2000. The Large, the Small and the Human Mind. *Cambridge* (2000).
- [45] Bjorn Poonen. 2012. Undecidable problems: a sampler. (apr 2012), 28. <http://www-math.mit.edu/~poonen/papers/sampler.pdf>
- [46] Viera K. Proulx. 1993. Computer Science in Elementary and Secondary Schools. In *Informatics and Changes in Learning, Proceedings of the IFIP TC3/WG3.1/WG3.5 Open Conference on Informatics and Changes in Learning, Gmunden, Austria, 7–11 June, 1993*. North-Holland, 95–101. <https://dblp.org/rec/bib/conf/ifip3-1/Proulx93>
- [47] James Purcell, Zhi Li, and Toby Cubitt. 2024. Chaitin Phase Transition. *arXiv:2410.02600 [quant-ph]* <https://arxiv.org/abs/2410.02600>
- [48] Daniel Richardson. 1968. Some undecidable problems involving elementary functions of a real variable. *The Journal of Symbolic Logic* 33, 4 (1968), 514–520. <https://doi.org/10.2307/2271358>
- [49] Gabriel Robins. 1988. Teaching Theoretical Computer Science at the Undergraduate Level. *University of California. CS Department* 88, 63 (02 1988).
- [50] Benjamin Schumacher. 2010. *Impossible: Physics Beyond the Edge*. The Teaching Company, Chantilly, VA. Course lecture series.
- [51] Fouriest Series. 2014. Chaos and the Double Pendulum. <https://fouriestseries.tumblr.com/post/8625333743/chaos-and-the-double-pendulum>. Accedido el 10 de abril de 2026.
- [52] Michael Sipser. 2012. *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning, Boston, MA.
- [53] John Stillwell. 2018. *Reverse Mathematics: Proofs from the Inside Out*. Princeton University Press.
- [54] John C. Stillwell. 2010. *Roads to Infinity: The Mathematics of Truth and Proof*. CRC Press.
- [55] Terence Tao. 2022. Almost all orbits of the Collatz map attain almost bounded values. *Forum of Mathematics, Pi* 8 (2022). <https://doi.org/10.1017/fmp.2020.2>
- [56] Jared Wasserman. 2016. Tiling Polyominoes Game. *Wolfram Demonstrations Project* (2016). demonstrations.wolfram.com/TilingPolyominoesGame/
- [57] Jeannette M. Wing. 2006. Computational Thinking. *Commun. ACM* 49, 3 (2006), 33–35.
- [58] Noson S. Yanofsky. 2016. *The Outer Limits of Reason: What Science, Mathematics, and Logic Cannot Tell Us*. The MIT Press.
- [59] Enrique Zeleny. 2014. Tiling with Pentominoes. *Wolfram Demonstrations Project* (2014). demonstrations.wolfram.com/TilingWithPentominoes/
- [60] Álvaro Perales-Eceiza, Toby Cubitt, Mile Gu, David Pérez-García, and Michael M. Wolf. 2024. Undecidability in Physics: A Review. *arXiv:2410.16532*.